Phase 1 Report: DOA-Sector Inference with LIF-Inspired Edge Processing

Hossein Molhem
Kennesaw State University
EE 6900 Embedded AI

July 3, 2026

Abstract

This report documents Phase 1 of an embedded-AI project focused on lightweight spike-based inference for edge sensing. A complete Python reference prototype was developed for direction-of-arrival (DOA) sector classification using a physics-based uniform linear array (ULA) signal model. The processing chain includes covariance estimation, beamspace feature extraction, feature normalization, nearest-centroid baseline classification, rate-based spike encoding, and leaky-integrate-and-fire (LIF)-inspired temporal inference. The nearest-centroid baseline achieved an overall test accuracy of 0.869048. A fixed LIF configuration achieved 0.825000 accuracy with a mean stable-decision latency of 6.423810 time steps. A later LIF parameter sweep improved the best LIF accuracy to 0.841667 at $T = 40$ and leak $= 0.99$. These results demonstrate a clear and controllable accuracy–latency trade-off, which is central to the embedded AI motivation of the project. The spike-based path is not presented as an accuracy-improving replacement for the continuous baseline; its added value is architectural, converting continuous beamspace classification into a sparse temporal inference process with measurable stable-decision latency and tunable accuracy–latency behavior.

Contents

1 Purpose and Scope **3**

2 Theoretical Background **3**
2.1 Uniform Linear Array Signal Model 3
2.2 Sample Covariance Matrix 3
2.3 Beamspace Feature Extraction 4
2.4 Nearest-Centroid Classification 4
2.5 Rate-Based Spike Encoding 4
2.6 LIF-Inspired Temporal Inference 4
2.7 Stable-Decision Latency 5

3 Signal Model and Dataset **5**

4 Beamspace Features **5**

5 Nearest-Centroid Baseline **6**

6 Spike Encoding and LIF-Inspired Inference **7**

7 Accuracy and Latency Analysis **7**

8 Run 005 Parameter Sweep **9**

9 Engineering Interpretation **10**

10 Conclusion **10**

A Additional Theoretical Details **11**
A.1 ULA Steering Vector and Phase Progression 11
A.2 Beamspace Energy Interpretation 11
A.3 Nearest-Centroid Rule as a Linear Score 11
A.4 Spike Encoding Statistics 12
A.5 LIF Leak and Latency Interpretation 12

References **13**

1 Purpose and Scope

The purpose of Phase 1 is to establish a complete and reproducible Python reference model for DOA-sector inference using a physics-based sensing model and a hardware-aware spike-based inference path. This phase is not intended to introduce a new high-resolution DOA algorithm. Instead, it provides an engineering baseline that can later be refactored toward embedded execution, PYNQ deployment, or Vitis HLS acceleration. This report is prepared as part of the EE 6900 Embedded AI project deliverable sequence [1].

The completed prototype connects three technical layers:

1. array signal processing through a ULA signal model and covariance-derived beamspace features,
2. machine-learning baseline evaluation through nearest-centroid classification,
3. sparse temporal inference through rate-based spike encoding and LIF-inspired evidence accumulation.

The complete Phase 1 source code, generated metrics, figures, and compiled deliverables are maintained in the project repository [2].

2 Theoretical Background

This section summarizes the theoretical foundation used in Phase 1. The purpose is to keep the main report self-contained while avoiding unnecessary derivation detail in the main body. The array-processing formulation follows standard ULA and covariance-based signal-processing theory [3,4]. The classification baseline follows classical pattern-recognition methodology [5], while the spike-based temporal inference path is motivated by spiking-neuron and neuromorphic-computing models [6–8]. Additional mathematical details are provided in the Appendix.

2.1 Uniform Linear Array Signal Model

A uniform linear array consists of N equally spaced sensor elements. For a narrowband plane wave arriving from angle θ, the relative phase progression across the array is determined by the element spacing d, the wavelength λ, and the wavenumber $k = 2\pi/\lambda$. With a broadside angle convention, a typical ULA steering vector can be written as [3]

$$a(\theta) = \begin{bmatrix} 1 & e^{-jkd\sin\theta} & e^{-j2kd\sin\theta} & \cdots & e^{-j(N-1)kd\sin\theta} \end{bmatrix}^T. \tag{1}$$

For the Phase 1 prototype, the exact objective is not continuous-angle estimation. Instead, the steering-vector model is used to generate physically meaningful array snapshots for a four-sector DOA classification task.

2.2 Sample Covariance Matrix

For each simulated source angle and SNR value, the received snapshot matrix is modeled as

$$X = a(\theta)s + N, \tag{2}$$

where $a(\theta)$ is the steering vector, s is the source waveform, and N is additive complex noise. The sample covariance matrix is

$$R_{xx} = \frac{1}{M} X X^H, \tag{3}$$

where M is the number of snapshots. In expectation, the covariance structure contains spatial information through the rank-one signal component associated with $a(\theta)a^H(\theta)$ and a noise contribution determined by the SNR [3, 4].

2.3 Beamspace Feature Extraction

Beamspace processing maps array-domain information into a set of angular responses. For a steering vector $b(\phi)$ evaluated at grid angle ϕ, a simple beamspace energy feature can be represented as

$$p(\phi) = b^H(\phi) R_{xx} b(\phi). \tag{4}$$

Evaluating $p(\phi)$ on a fixed angular grid produces a compact real-valued feature vector. In Phase 1, this feature vector is used for coarse sector classification rather than fine DOA estimation [3].

2.4 Nearest-Centroid Classification

The non-spiking baseline uses a nearest-centroid decision rule. If x is a normalized feature vector and μ_c is the training-set centroid for class c, the baseline decision is

$$\hat{c} = \arg\min_c \|x - \mu_c\|_2^2. \tag{5}$$

This classifier is intentionally simple and interpretable. It provides a useful reference for measuring how much accuracy is lost when continuous-valued features are converted into sparse spike trains [5].

2.5 Rate-Based Spike Encoding

After normalization, each feature component is mapped to a spike probability. For feature component $z_i \in [0, 1]$, the spike at time step t can be modeled as

$$s_i[t] \sim \text{Bernoulli}(z_i). \tag{6}$$

The time-average spike count approximates the original normalized feature value:

$$\mathbb{E}\left[\frac{1}{T}\sum_{t=1}^{T} s_i[t]\right] = z_i. \tag{7}$$

This representation introduces stochastic quantization error but creates a sparse temporal signal appropriate for hardware-facing inference experiments [6, 8].

2.6 LIF-Inspired Temporal Inference

The LIF-inspired classifier accumulates spike-driven evidence over time. For class c, the membrane evidence follows

$$v_c[t] = \alpha v_c[t-1] + i_c[t], \tag{8}$$

where α is the leak factor and $i_c[t]$ is the spike-driven input current. A higher leak value preserves past evidence more strongly, while a shorter time window reduces latency but may reduce classification accuracy. This leaky temporal accumulation is a simplified hardware-facing abstraction of LIF-style dynamics [7, 8].

2.7 Stable-Decision Latency

The classifier produces a class decision at each time step by selecting the largest membrane value:

$$d[t] = \arg\max_c v_c[t]. \tag{9}$$

The stable-decision latency is defined as the earliest time step after which the decision remains equal to the final decision. This metric provides a practical way to evaluate whether the temporal classifier can support early-exit or adaptive-latency edge inference [8].

3 Signal Model and Dataset

The Phase 1 dataset is generated using a narrowband single-source ULA model. For each source angle θ and SNR value, the received array snapshot matrix is modeled as

$$X = a(\theta)s + N, \tag{10}$$

where $a(\theta)$ is the ULA steering vector, s is a complex narrowband source signal, and N is circular complex additive white Gaussian noise. The sample covariance matrix is computed as

$$R_{xx} = \frac{1}{M} X X^H, \tag{11}$$

where $M = 128$ snapshots. The generated dataset and resulting artifacts are part of the reproducible Phase 1 implementation [2].

A compact beamspace feature vector is extracted from the covariance matrix by evaluating spatial energy over a fixed angular grid. The features are normalized using training-set statistics before classification or spike encoding.

Table 1: Phase 1 Dataset Configuration

Parameter	Value
Array type	Uniform Linear Array
Number of array elements	8
Snapshots per sample	128
Element spacing	$\lambda/2$
Source count	1
DOA range	-60° to $+60^\circ$
Number of DOA sectors	4
SNR values	$-10, -5, 0, 5, 10, 15, 20$ dB
Samples per sector per SNR	100
Total samples	2800
Train/test split	70/30 stratified
Test samples	840

4 Beamspace Features

Beamspace features provide a compact representation of angular information. The objective is not to estimate the exact continuous DOA angle, but to classify the received signal into one of four

angular sectors. A beamspace representation is appropriate for this task because it reduces the complex covariance structure into a small real-valued feature vector that captures spatial energy distribution [3].

Figure 1 shows an example beamspace response generated during the reference run.

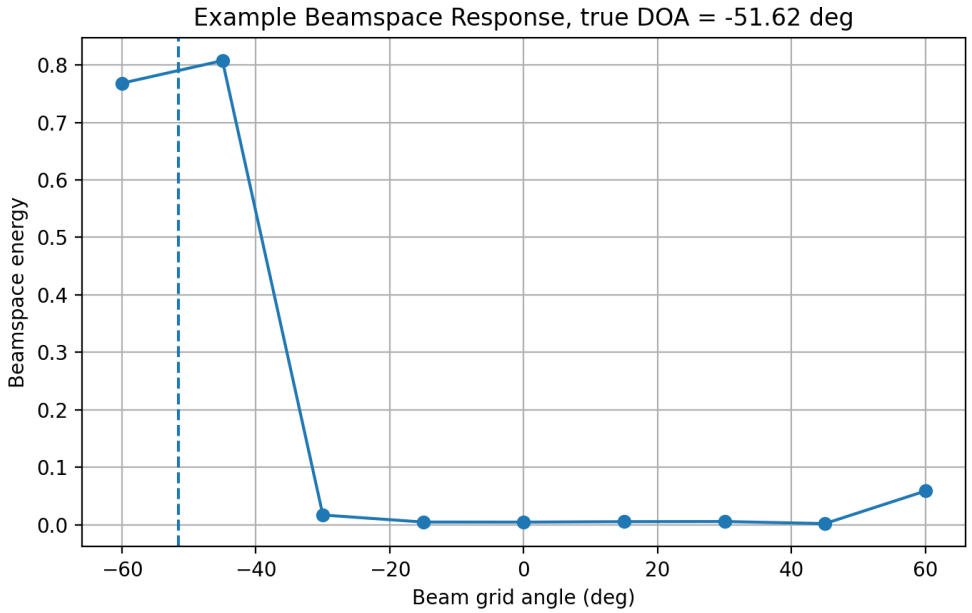


Figure 1: Example beamspace response from the Phase 1 ULA simulation.

5 Nearest-Centroid Baseline

The non-spiking reference classifier is a nearest-centroid classifier trained on the normalized beamspace features. This classifier is deterministic, interpretable, and lightweight. Its purpose is to establish the amount of DOA-sector information available in the compact feature vector before converting the features into spike trains [5].

The nearest-centroid baseline achieved

$$\text{Accuracy}_{\text{baseline}} = 0.869048. \tag{12}$$

Table 2: Nearest-Centroid Accuracy Versus SNR

SNR (dB)	Accuracy
-10	0.758333
-5	0.908333
0	0.891667
5	0.858333
10	0.866667
15	0.925000
20	0.875000

6 Spike Encoding and LIF-Inspired Inference

Normalized beamspace features are converted into binary spike trains using rate-based encoding. A larger normalized feature value corresponds to a higher probability of spike emission across the time window. This representation is sparse and temporal, making it a useful abstraction for later hardware-oriented experiments [6, 8].

The LIF-inspired classifier is implemented as a class-level leaky membrane accumulator. It is not a biologically detailed SNN. Instead, it is a hardware-facing approximation in which each class accumulates spike-driven evidence over time. The membrane state can be interpreted as

$$v_c[t] = \alpha v_c[t-1] + i_c[t], \tag{13}$$

where $v_c[t]$ is the membrane value for class c, α is the leak factor, and $i_c[t]$ is the spike-driven input current at time step t [7].

The fixed Run 003 configuration used

$$T = 20, \qquad \alpha = 0.97. \tag{14}$$

It achieved

$$\text{Accuracy}_{\text{LIF, fixed}} = 0.825000, \tag{15}$$

with mean stable-decision latency

$$\bar{L} = 6.423810 \text{ time steps}. \tag{16}$$

7 Accuracy and Latency Analysis

Run 004 generated report-ready figures comparing the nearest-centroid baseline and the LIF-inspired classifier across the same SNR grid. Figure 2 compares accuracy versus SNR. Figure 3 shows mean stable-decision latency versus SNR.

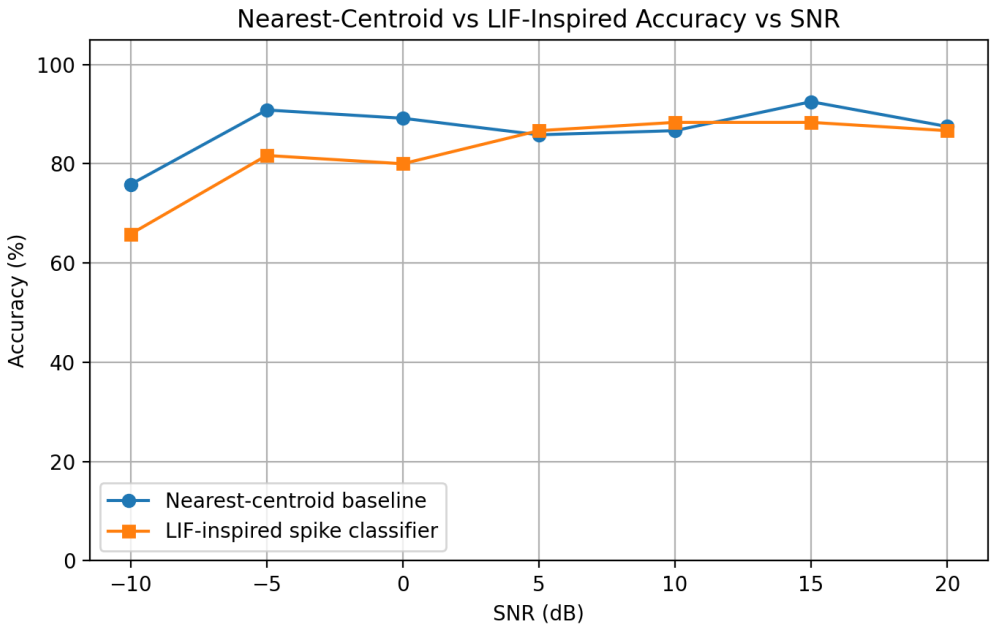


Figure 2: Accuracy comparison between the nearest-centroid baseline and the LIF-inspired classifier.

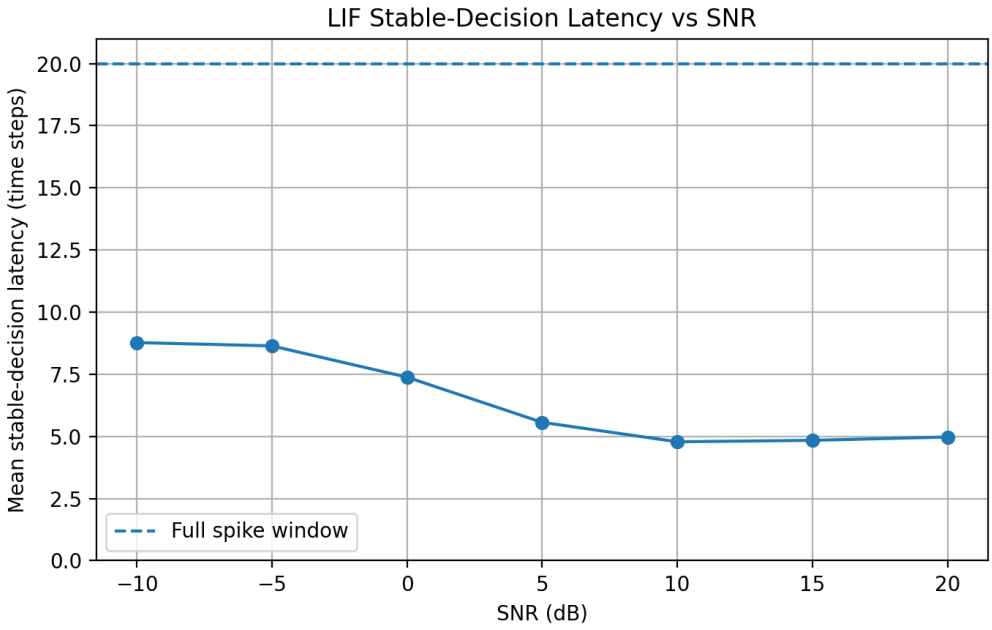


Figure 3: Mean stable-decision latency of the LIF-inspired classifier versus SNR.

The latency trend supports the edge-inference interpretation. When the signal quality is higher, the LIF decision generally stabilizes earlier. This suggests that temporal inference can support early-exit or adaptive-latency decision logic in later embedded implementations [8].

8 Run 005 Parameter Sweep

Run 005 evaluated the sensitivity of the LIF-inspired classifier to spike-window length and membrane leak factor. The sweep grid was

$$T \in \{10, 20, 30, 40\}, \qquad \alpha \in \{0.90, 0.95, 0.97, 0.99\}. \tag{17}$$

This created 16 configurations.

Table 3: Run 005 Key Results

Case	T	Leak	Accuracy	Mean latency
Best LIF accuracy	40	0.99	0.841667	8.396429
Lowest latency	10	0.99	0.776190	3.782143
Fixed Run 003	20	0.97	0.825000	6.423810
Non-spiking baseline	–	–	0.869048	–

Table 4: Full Run 005 LIF Parameter Sweep

T	Leak	Accuracy	Mean latency
10	0.90	0.773810	4.280952
10	0.95	0.778571	4.011905
10	0.97	0.779762	3.914286
10	0.99	0.776190	3.782143
20	0.90	0.809524	7.732143
20	0.95	0.816667	6.579762
20	0.97	0.819048	6.340476
20	0.99	0.816667	5.753571
30	0.90	0.802381	10.946429
30	0.95	0.829762	8.870238
30	0.97	0.838095	8.126190
30	0.99	0.832143	7.352381
40	0.90	0.801190	14.945238
40	0.95	0.822619	10.778571
40	0.97	0.830952	9.697619
40	0.99	0.841667	8.396429

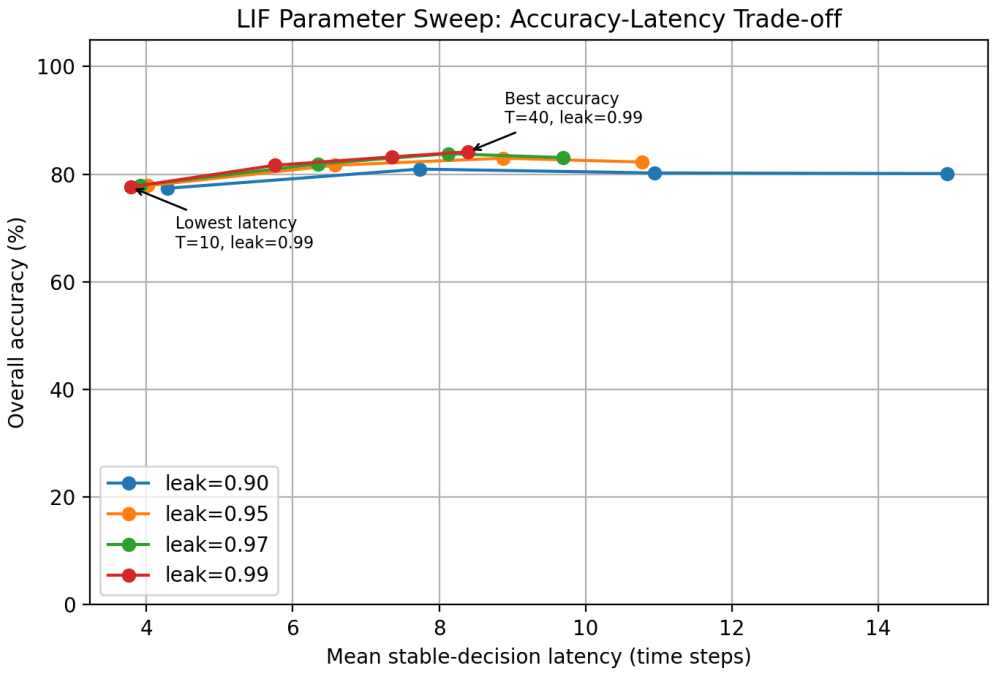


Figure 4: Run 005 LIF accuracy–latency trade-off across time-window and leak configurations.

9 Engineering Interpretation

The parameter sweep confirms a clear accuracy–latency design trade-off. Increasing the spike time window generally improves classification accuracy because the rate-coded representation has more time steps to approximate the original continuous feature vector. However, a longer time window can increase temporal budget and stable-decision latency.

The leak factor controls evidence retention. Higher leak values preserve accumulated information better. In this prototype, the best LIF accuracy occurs at $T = 40$ and leak $= 0.99$. The best LIF result improves over the fixed Run 003 configuration but remains below the continuous nearest-centroid baseline. This result is expected and useful: spike-based inference trades some continuous-valued accuracy for sparse temporal processing and explicit latency control [8].

10 Conclusion

The Phase 1 Python prototype successfully establishes an end-to-end DOA-sector inference pipeline based on a physics-derived ULA signal model. The nearest-centroid classifier demonstrates that compact beamspace features provide strong sector-level DOA information, achieving 86.9% test accuracy.

The spike-based path did not improve raw classification accuracy over the continuous nearest-centroid baseline. The baseline achieved 86.9% accuracy, while the fixed LIF-inspired configuration achieved 82.5%, and the best LIF configuration from the parameter sweep achieved 84.17%. However, the value of the spike-based formulation is architectural rather than purely accuracy-based. It converts continuous beamspace features into sparse temporal spike trains, enables leaky evidence accumulation, exposes stable-decision latency as a measurable output, and creates a tunable

accuracy–latency design space through the spike time window and leak factor.

Therefore, the Phase 1 spike-based result should be interpreted as a hardware-aware embedded-AI baseline, not as an accuracy-improving replacement for the continuous classifier. Its main contribution is the transformation of DOA-sector inference into a sparse temporal decision process suitable for later neuromorphic, PYNQ, FPGA, or Vitis HLS-oriented exploration. The source code and generated Phase 1 artifacts are preserved for reproducibility in the project repository [2].

A Additional Theoretical Details

A.1 ULA Steering Vector and Phase Progression

For a ULA with inter-element spacing d, the path-length difference between adjacent sensors is $d \sin\theta$ under the broadside convention. The corresponding phase shift is

$$\psi(\theta) = kd \sin\theta = \frac{2\pi d}{\lambda} \sin\theta. \tag{18}$$

For half-wavelength spacing, $d = \lambda/2$, this becomes

$$\psi(\theta) = \pi \sin\theta. \tag{19}$$

This phase progression is the physical basis for the steering vector used to generate DOA-dependent array snapshots [3].

A.2 Beamspace Energy Interpretation

The beamspace response

$$p(\phi) = b^H(\phi) R_{xx} b(\phi) \tag{20}$$

can be interpreted as the output power of a conventional beam steered toward grid angle ϕ. If the grid steering vector is aligned with the true source direction, the response tends to be larger. Therefore, the vector of grid responses provides a compact angular signature for sector classification [3].

A.3 Nearest-Centroid Rule as a Linear Score

The nearest-centroid decision rule can be expanded as

$$\hat{c} = \arg\min_c \|x - \mu_c\|_2^2 \tag{21}$$

$$= \arg\min_c \left(x^T x - 2\mu_c^T x + \mu_c^T \mu_c \right). \tag{22}$$

Since $x^T x$ does not depend on class, the equivalent decision is

$$\hat{c} = \arg\max_c \left(2\mu_c^T x - \|\mu_c\|_2^2 \right). \tag{23}$$

This form is useful for the LIF-inspired implementation because it can be interpreted as class evidence accumulation using class-dependent weights and biases [5].

A.4 Spike Encoding Statistics

For rate-based encoding, the spike count for feature i over T time steps is

$$K_i = \sum_{t=1}^{T} s_i[t]. \tag{24}$$

If $s_i[t] \sim \text{Bernoulli}(z_i)$, then

$$\mathbb{E}[K_i] = Tz_i, \tag{25}$$

$$\text{Var}(K_i) = Tz_i(1 - z_i). \tag{26}$$

Therefore, a longer spike window reduces relative stochastic error but increases the temporal budget [6, 7].

A.5 LIF Leak and Latency Interpretation

The LIF-inspired update

$$v_c[t] = \alpha v_c[t-1] + i_c[t] \tag{27}$$

acts as a leaky temporal integration of evidence. When α is close to one, previous evidence is retained for more time steps. When α is smaller, the classifier reacts more strongly to recent spikes but may lose accumulated evidence more quickly. This explains why the Phase 1 parameter sweep uses both T and α as design variables [7, 8].

References

[1] Kennesaw State University, "Ee 6900 embedded ai: Phase 1 project deliverable," Course project report, 2026.

[2] H. Molhem, "Embedded ai snn edge inference: Phase 1 python prototype," GitHub repository, 2026, accessed: 2026-07-03. [Online]. Available: https://github.com/hmolhem/embedded-ai-snn-edge-inference

[3] H. L. V. Trees, *Optimum Array Processing: Part IV of Detection, Estimation, and Modulation Theory*. New York, NY: John Wiley & Sons, 2002.

[4] S. Haykin, *Adaptive Filter Theory*, 5th ed. Upper Saddle River, NJ: Pearson, 2014.

[5] R. O. Duda, P. E. Hart, and D. G. Stork, *Pattern Classification*, 2nd ed. New York, NY: John Wiley & Sons, 2001.

[6] W. Maass, "Networks of spiking neurons: The third generation of neural network models," *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.

[7] W. Gerstner and W. M. Kistler, *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge, UK: Cambridge University Press, 2002.

[8] K. Roy, A. Jaiswal, and P. Panda, "Towards spike-based machine intelligence with neuromorphic computing," *Nature*, vol. 575, pp. 607–617, 2019.